

SUPSI

Magic Eraser

Studente/i

Abate Luca

Vavassori Enrico

Relatore

Gremlich Giuliano

Correlatore

Quattrini Andrea

Committente

PLACEHOLDER

Corso di laurea

Ingegneria Informatica TP

Modulo

Progetto di Semestre

Anno

2026

Data

28 aprile 2026

Indice

Abstract	iii
Introduzione	v
1 Motivazione e Contesto	1
1.1 Rilevanza del problema	1
1.2 Contesto tecnologico	1
1.3 Obiettivo del progetto	1
2 Definizione del Problema	3
2.1 Problema pratico	3
2.2 Limiti delle soluzioni attuali	3
2.3 Bisogni dell'utente	3
3 Stato dell'Arte	4
3.1 Image Inpainting	4
3.2 Segmentazione e maschere	4
3.3 Modelli generativi	4
3.4 Confronto con strumenti esistenti	5
4 Approccio alla Soluzione	6
4.1 Descrizione generale	6
4.2 Architettura del sistema	6
4.2.1 Frontend	6
4.2.2 Backend	6
4.2.3 Database	7
4.2.4 AI provider	7
4.3 Pipeline di elaborazione	7
4.4 Scelte tecnologiche	7
5 Implementazione	8
5.1 Struttura del sistema	8

5.2	Frontend	8
5.3	Backend e API	8
5.4	Gestione immagini e progetti	9
5.5	Pipeline AI	9
5.5.1	Segmentazione	9
5.5.2	Rimozione	9
5.5.3	Generazione	9
5.6	Organizzazione del codice	9
6	Risultati e Validazione	10
6.1	Funzionalità implementate	10
6.2	Test	10
6.3	Validazione	10
6.4	Risultati	11
7	Conclusioni	12
7.1	Valutazione del lavoro	12
7.2	Limiti della soluzione	12
7.3	Contributo	12
7.4	Sviluppi futuri	13
A	Manuale Utente	15
A.1	Login	15
A.2	Creare un progetto	15
A.3	Caricare un'immagine	15
A.4	Usare il laboratorio	15
A.5	Salvare una pipeline	15
B	API principali	16
B.1	Elenco endpoint	16
C	Configurazione e Avvio	17
C.1	Docker Compose	17
C.2	Variabili ambiente	17
C.3	Avvio frontend e backend	17

Abstract

Italiano

Magic Eraser è una piattaforma web per l'editing di immagini assistito da intelligenza artificiale. Il progetto nasce nel contesto degli strumenti generativi applicati alla computer vision, con particolare attenzione alla rimozione automatica di oggetti e alla modifica di immagini tramite maschere. Il problema affrontato riguarda la difficoltà, per utenti non esperti, di rimuovere contenuti indesiderati senza ricorrere a software professionali complessi e senza dover costruire manualmente selezioni precise.

La soluzione proposta combina gestione dei progetti, caricamento delle immagini, segmentazione da prompt, rimozione tramite maschera e generazione di nuove aree coerenti con il contesto visivo. Il sistema è organizzato come applicazione web con frontend React, backend FastAPI, persistenza su Supabase Postgres, storage delle immagini e integrazione con provider AI esterni. L'elaborazione è strutturata in pipeline, così da rendere tracciabili i passaggi applicati e consentire all'utente di salvare, rivedere e riutilizzare i risultati.

Il lavoro ha portato alla realizzazione di un prototipo funzionante che permette di creare progetti, caricare immagini, applicare processi AI e mantenere una cronologia ordinata delle elaborazioni. I risultati mostrano come l'approccio semplifica il flusso di editing e rende accessibili operazioni avanzate, pur però rimanendo dipendente dalla qualità dei modelli esterni, dalla precisione delle maschere e dai tempi di risposta dei servizi AI. In conclusione, Magic Eraser rappresenta una base estendibile per sperimentare workflow di image editing generativo e per confrontare modelli differenti in situazioni reali.

English

Magic Eraser is a web platform for AI-assisted image editing. The project was born in the context of generative tools applied to computer vision, with a particular focus on automatic object removal and image editing using masks. The problem addressed is the difficulty for non-expert users to remove unwanted content without resorting to complex professional software and without having to manually construct precise selections.

The proposed solution combines project management, image loading, segmentation from prompts, mask removal, and generation of new areas consistent with the visual context. The system is organized as a web application with a React frontend, FastAPI backend, persistence on Supabase Postgres, image storage, and integration with external AI providers. Processing is structured in pipelines, making the applied steps traceable and allowing the user to save, review, and reuse the results.

The work led to the creation of a working prototype that allows you to create projects, upload images, apply AI processes, and maintain an organized history of processing. The results show that the approach simplifies the editing workflow and makes advanced operations accessible, while still being dependent on the quality of external models, mask accuracy, and AI service response times. In conclusion, Magic Eraser represents an extensible basis for experimenting with generative image editing workflows and comparing different models in real-world situations.

Introduzione

Magic Eraser è una piattaforma web per l'editing di immagini assistito da AI. Il sistema consente di organizzare immagini in progetti, applicare processi di segmentazione, rimozione e generazione, e conservare le pipeline di elaborazione prodotte durante il lavoro. L'obiettivo non è soltanto rimuovere oggetti indesiderati, ma costruire un ambiente sperimentale in cui l'utente possa osservare, ripetere e confrontare il comportamento di modelli generativi applicati a immagini reali.

Struttura del documento

Il documento segue il percorso logico del progetto: prima chiarisce il contesto e il problema, poi descrive le tecnologie e gli approcci esistenti, quindi presenta l'architettura, l'implementazione e i risultati ottenuti. Le appendici raccolgono informazioni operative utili per usare il sistema, consultare le API e configurare l'ambiente tecnico.

Guida alla lettura

Il documento è organizzato in sette capitoli principali. Il Capitolo 1 introduce la motivazione del progetto, l'importanza del problema e il contesto tecnologico. Il Capitolo 2 definisce il problema pratico, i limiti delle soluzioni attuali e le necessità dell'utente. Il Capitolo 3 presenta lo stato dell'arte relativo a inpainting, segmentazione, maschere e modelli generativi. Il Capitolo 4 descrive la soluzione proposta, l'architettura del sistema, la pipeline di elaborazione e le scelte tecnologiche. Il Capitolo 5 approfondisce l'implementazione, includendo frontend, backend, gestione immagini e pipeline AI. Il Capitolo 6 riporta funzionalità implementate, test, validazione e risultati. Il Capitolo 7 conclude il lavoro con una valutazione critica, i limiti della soluzione, il contributo del progetto e gli sviluppi futuri.

Capitolo 1

Motivazione e Contesto

La diffusione di strumenti di image editing basati su AI ha reso più accessibili operazioni che in passato richiedevano competenze grafiche avanzate. La rimozione di oggetti da immagini, spesso indicata come funzionalità di “magic eraser”, è un caso particolarmente rilevante perché unisce comprensione semantica della scena, generazione di maschere e ricostruzione visiva delle aree modificate.

1.1 Rilevanza del problema

L'editing tradizionale richiede selezioni manuali, conoscenza di strumenti professionali e tempo per rifinire il risultato. In molti scenari, come la preparazione di immagini per presentazioni, contenuti web, materiale didattico o prototipi visivi, l'utente ha invece bisogno di un flusso rapido e guidato. Un sistema automatico di rimozione oggetti può ridurre la complessità operativa, lasciando all'utente il controllo sull'intenzione e delegando ai modelli AI le operazioni più tecniche.

1.2 Contesto tecnologico

Il progetto si colloca all'intersezione tra AI generativa e computer vision. La computer vision permette di individuare aree rilevanti dell'immagine tramite segmentazione e maschere, mentre i modelli generativi consentono di ricostruire contenuti coerenti nelle zone rimosse. L'integrazione di questi elementi in una piattaforma web richiede anche aspetti applicativi: gestione degli utenti, persistenza dei dati, archiviazione delle immagini, tracciamento delle elaborazioni e interfacce semplici da usare.

1.3 Obiettivo del progetto

L'obiettivo generale è sviluppare un sistema web che permetta di caricare immagini, selezionare processi AI, generare maschere, rimuovere contenuti e salvare i risultati all'inter-

no di progetti organizzati. Il sistema deve essere sufficientemente semplice per un utente non specialista, ma anche strutturato in modo da supportare analisi e sperimentazione sui modelli impiegati.

Capitolo 2

Definizione del Problema

Il problema principale riguarda la trasformazione di un'attività grafica complessa in un workflow accessibile. La rimozione di un oggetto non consiste solo nel cancellare pixel: occorre identificare correttamente l'area interessata, produrre una maschera utilizzabile, inviare l'immagine a un modello adeguato e valutare se il risultato è coerente con il contesto.

2.1 Problema pratico

L'utente vuole modificare un'immagine senza dover conoscere strumenti professionali. Nel caso di Magic Eraser, il flusso pratico comprende tre azioni principali: scegliere un'immagine, indicare cosa rimuovere o generare, e ottenere un output visualmente credibile. La generazione delle maschere è un passaggio centrale perché determina l'area su cui agisce il modello di rimozione o riempimento.

2.2 Limiti delle soluzioni attuali

Le soluzioni tradizionali offrono controllo elevato, ma richiedono competenze specifiche. Alcuni strumenti automatici semplificano l'operazione, ma spesso nascondono completamente la pipeline, non permettono di salvare i passaggi intermedi o non consentono di confrontare modelli e impostazioni. Inoltre, l'utente può avere difficoltà a organizzare più immagini, versioni e risultati all'interno dello stesso lavoro.

2.3 Bisogni dell'utente

I bisogni individuati sono semplicità d'uso, automazione dei processi ripetitivi e gestione ordinata delle immagini. L'utente deve poter creare progetti, caricare più asset, avviare processi AI, visualizzare output intermedi e finali, e ritrovare le pipeline salvate. La piattaforma deve quindi comportarsi sia come editor assistito sia come ambiente di organizzazione del lavoro.

Capitolo 3

Stato dell'Arte

Lo stato dell'arte considera le tecniche e gli strumenti che rendono possibile l'editing generativo: image inpainting, segmentazione, maschere e modelli prompt-based. Questi concetti costituiscono la base teorica e applicativa della soluzione sviluppata.

3.1 Image Inpainting

L'Image inpainting è il processo di ricostruzione di aree mancanti, danneggiate o mascherate di un'immagine. Nei sistemi moderni, l'inpainting non si limita alla copia di texture circostanti, ma utilizza modelli generativi capaci di interpretare il contesto visivo. Nel progetto, questa tecnica è utilizzata per ottenere risultati coerenti dopo la rimozione di un oggetto o per riempire una regione secondo un prompt.

3.2 Segmentazione e maschere

La segmentazione consente di individuare regioni dell'immagine corrispondenti a oggetti o concetti indicati dall'utente. Il risultato viene rappresentato tramite maschere, immagini binarie o semi-binarie che separano le zone da modificare dal resto della scena. Nel workflow di Magic Eraser, la maschera è l'elemento di collegamento tra la selezione semantica e l'operazione generativa successiva.

3.3 Modelli generativi

I modelli generativi prompt-based permettono di guidare il risultato tramite descrizioni testuali. Le tecniche text-to-image producono immagini partendo da testo, mentre le tecniche image-to-image trasformano un'immagine esistente mantenendo parte della sua struttura. Per l'editing con maschera, l'approccio più rilevante è quello che combina immagine sorgente, maschera e prompt opzionale, così da modificare solo una porzione controllata dell'immagine.

3.4 Confronto con strumenti esistenti

Strumento	Analisi sintetica
Editor professionali	Offrono controllo preciso e strumenti avanzati, ma richiedono competenze specifiche. Magic Eraser privilegia automazione e semplicità d'uso.
Strumenti automatici	Consentono una rimozione rapida degli oggetti, ma spesso rendono poco visibile la pipeline. Magic Eraser conserva passaggi e risultati.
Demo di modelli AI	Permettono di provare singoli modelli, ma offrono poca gestione di progetti e immagini. Magic Eraser integra i modelli in un'applicazione completa.

Capitolo 4

Approccio alla Soluzione

La soluzione proposta è una piattaforma web composta da interfaccia utente, API backend, database, storage immagini e integrazione con provider AI. L'architettura separa le responsabilità principali, rendendo il sistema più manutenibile e predisposto all'estensione con nuovi modelli o nuovi processi.

4.1 Descrizione generale

L'utente accede all'applicazione, crea o seleziona un progetto, carica immagini e apre il laboratorio di editing. Nel laboratorio può aggiungere processi alla pipeline, scegliere il modello, impostare prompt o parametri aggiuntivi e generare un risultato. Le pipeline vengono salvate con i relativi step, input, output, maschere e stato di elaborazione.

4.2 Architettura del sistema

4.2.1 Frontend

Il frontend è sviluppato con React e Vite. Le pagine principali includono login, signup, home, gallery, laboratory, pipelines e profile. I componenti gestiscono la navigazione, la visualizzazione dei progetti, l'upload delle immagini, la configurazione dei processi e la consultazione delle pipeline salvate.

4.2.2 Backend

Il backend è sviluppato con FastAPI e fornisce endpoint REST per progetti, immagini, profilo utente, processi AI e pipeline di laboratorio. La logica applicativa è organizzata in servizi, repository, schemi di validazione e router API.

4.2.3 Database

La persistenza è basata su Supabase Postgres. Le entità principali sono `Profile`, `Project`, `Image`, `LaboratoryPipeline` e `LaboratoryPipelineStep`. Questa struttura consente di collegare utenti, progetti, immagini sorgente e cronologie di elaborazione.

4.2.4 AI provider

L'integrazione con i modelli AI è gestita tramite registry e adapter. Il catalogo dei processi espone tre famiglie principali: segmentazione da prompt, rimozione con maschera e generazione da prompt. Gli adapter traducono le richieste interne nel formato richiesto dal provider esterno.

4.3 Pipeline di elaborazione

La pipeline rappresenta la sequenza dei processi applicati a un'immagine. Uno step può generare una maschera, uno step successivo può usare quella maschera per rimuovere un oggetto, e un ulteriore step può generare contenuto tramite prompt. Ogni step mantiene input, output, tipo di processo, modello scelto, impostazioni aggiuntive e stato.

4.4 Scelte tecnologiche

React e Vite sono stati scelti per costruire un'interfaccia reattiva e modulare. FastAPI è stato adottato per la chiarezza nella definizione delle API, la validazione tramite schemi e la facilità di integrazione con servizi Python. Supabase Postgres fornisce un database remoto adatto alla persistenza dei dati applicativi, mentre lo storage immagini permette di mantenere file sorgente e risultati accessibili dal frontend e dai provider AI.

Capitolo 5

Implementazione

L'implementazione segue una separazione netta tra frontend, backend, persistenza e integrazione AI. Questa organizzazione permette di modificare una parte del sistema senza compromettere le altre e rende più semplice aggiungere nuovi processi.

5.1 Struttura del sistema

Il progetto è diviso principalmente in `frontend/` e `backend/`. Il frontend contiene pagine, componenti, hook, tipi TypeScript e client API. Il backend contiene router, servizi, repository, modelli, schemi, processi, registry dei modelli e utility per la gestione delle maschere.

5.2 Frontend

Le pagine di login e signup gestiscono l'accesso dell'utente. La home permette di creare progetti, caricare immagini, visualizzare anteprime e aprire il laboratorio. La gallery offre una visione organizzata degli asset. La pagina laboratory consente di costruire e applicare pipeline di editing. La pagina pipelines permette di consultare elaborazioni salvate, mentre la pagina profile raccoglie le informazioni dell'utente.

5.3 Backend e API

Gli endpoint principali sono organizzati nei router `projects`, `images`, `profile`, `processes` e `laboratory_pipelines`. Le richieste vengono validate tramite schemi, elaborate dai servizi applicativi e persistite tramite repository. Questa separazione riduce l'accoppiamento tra API, logica di business e accesso al database.

5.4 Gestione immagini e progetti

I progetti raggruppano le immagini caricate e le pipeline associate. L'upload salva il file nello storage e registra nel database le informazioni necessarie a recuperarlo. Le immagini possono essere usate come input per il laboratorio e gli output prodotti dai modelli possono essere salvati nello stesso contesto progettuale.

5.5 Pipeline AI

5.5.1 Segmentazione

Il processo `segment_from_prompt` riceve un'immagine e un prompt che descrive l'oggetto o l'area da individuare. Il risultato è una maschera utilizzabile negli step successivi.

5.5.2 Rimozione

Il processo `remove_with_mask` riceve immagine e maschera. Il modello elimina l'area selezionata e ricostruisce il contenuto circostante in modo coerente.

5.5.3 Generazione

Il processo `generate_from_prompt` permette di riempire una regione mascherata seguendo un prompt testuale. Questo consente non solo di cancellare contenuti, ma anche di sostituirli con elementi generati.

5.6 Organizzazione del codice

Il backend separa router API, servizi, repository, processi e registry dei modelli. Il frontend separa pagine, componenti riutilizzabili, hook e client HTTP. Questa struttura rende esplicite le responsabilità: l'interfaccia gestisce l'interazione utente, il backend coordina il workflow, i repository gestiscono la persistenza e gli adapter comunicano con i provider AI.

Capitolo 6

Risultati e Validazione

La validazione considera sia le funzionalità implementate sia la capacità del sistema di rispondere ai requisiti individuati. L'obiettivo è verificare che il prototipo permetta effettivamente di svolgere il workflow previsto: organizzare immagini, applicare processi AI e salvare risultati.

6.1 Funzionalità implementate

Il sistema implementa autenticazione, gestione del profilo, creazione e modifica di progetti, upload di immagini, consultazione della gallery, laboratorio di editing, catalogo dei processi AI, esecuzione di step, anteprima di maschere convex hull e salvataggio delle pipeline. Le funzionalità sono integrate in un flusso utente coerente, dalla selezione dell'immagine fino alla consultazione dei risultati.

6.2 Test

Il backend include test unitari e di integrazione per servizi, dipendenze di autenticazione e API principali. Il frontend include test su client API, hook, pagine e componenti del laboratorio. I test verificano casi come gestione delle risposte HTTP, progetti, profilo, pipeline, selezione di immagini e rendering dei componenti principali.

6.3 Validazione

Rispetto ai bisogni iniziali, la soluzione soddisfa i requisiti di semplicità, automazione e organizzazione. L'utente può eseguire operazioni complesse senza intervenire manualmente sui pixel, mentre il sistema conserva le informazioni necessarie a comprendere la sequenza di elaborazione applicata.

6.4 Risultati

I risultati attesi sono immagini prima/dopo in cui l'oggetto selezionato viene rimosso o sostituito. La qualità dipende da tre fattori principali: precisione della maschera, coerenza del modello generativo e complessità dello sfondo. Su sfondi uniformi il risultato tende a essere più stabile, mentre scene complesse, oggetti parzialmente occlusi o prompt ambigui possono produrre artefatti.

Capitolo 7

Conclusioni

Il progetto ha portato alla realizzazione di una piattaforma web capace di integrare gestione delle immagini e workflow AI per l'editing generativo. Magic Eraser dimostra come segmentazione, maschere e inpainting possano essere combinati in un ambiente accessibile e organizzato.

7.1 Valutazione del lavoro

Rispetto al problema iniziale, il sistema riduce la complessità dell'editing e rende disponibili operazioni avanzate attraverso un'interfaccia guidata. La scelta di salvare pipeline e step permette inoltre di analizzare il processo, non soltanto il risultato finale.

7.2 Limiti della soluzione

I principali limiti riguardano la dipendenza da provider AI esterni, la latenza delle richieste, i costi potenziali delle API e la variabilità dei risultati generativi. La qualità finale non è sempre prevedibile e può richiedere più tentativi, soprattutto quando la maschera non delimita correttamente l'area da modificare.

7.3 Contributo

Il contributo del progetto consiste nell'integrazione di funzionalità di editing AI in una piattaforma strutturata per progetti e pipeline. A differenza di strumenti più chiusi, Magic Eraser rende espliciti gli step e crea una base utile per sperimentare modelli, parametri e combinazioni di processi.

7.4 Sviluppi futuri

Gli sviluppi futuri includono un editor manuale delle maschere, confronto tra più modelli, funzionalità di undo e redo, esportazione delle pipeline, metriche quantitative sulla qualità, elaborazione batch, supporto a modelli locali, collaborazione multiutente, versionamento dei risultati e dashboard per monitorare tempi e costi di elaborazione.

Bibliografia

- [1] Kirillov, A. et al. Segment Anything. IEEE/CVF International Conference on Computer Vision, 2023.
- [2] Rombach, R. et al. High-Resolution Image Synthesis with Latent Diffusion Models. IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.
- [3] Suvorov, R. et al. Resolution-robust Large Mask Inpainting with Fourier Convolutions. IEEE/CVF Winter Conference on Applications of Computer Vision, 2022.
- [4] FastAPI Documentation. Framework Python per la costruzione di API web.
- [5] React Documentation. Libreria JavaScript per la costruzione di interfacce utente.
- [6] Supabase Documentation. Piattaforma backend basata su PostgreSQL, autenticazione e storage.

Appendice A

Manuale Utente

A.1 Login

L'utente accede tramite la pagina di login. In assenza di un account può utilizzare la pagina di signup per creare un nuovo profilo.

A.2 Creare un progetto

Nella home l'utente inserisce il nome del progetto e lo crea. Il progetto diventa il contenitore delle immagini caricate e delle pipeline generate.

A.3 Caricare un'immagine

L'utente seleziona uno o più file, sceglie il progetto di destinazione e avvia l'upload. Le immagini vengono mostrate come anteprime e possono essere aperte nel laboratorio.

A.4 Usare il laboratorio

Nel laboratorio l'utente sceglie il processo da applicare, seleziona il modello, imposta eventuali prompt o parametri e avvia l'elaborazione. Gli output intermedi possono diventare input degli step successivi.

A.5 Salvare una pipeline

Una pipeline può essere salvata con nome, immagine iniziale, immagine finale e lista degli step eseguiti. In questo modo il lavoro rimane consultabile nella sezione pipelines.

Appendice B

API principali

B.1 Elenco endpoint

- `/projects`: creazione, lettura, aggiornamento ed eliminazione dei progetti.
- `/images`: gestione delle immagini caricate.
- `/profile`: lettura e aggiornamento del profilo utente.
- `/processes/catalog`: catalogo dei processi AI disponibili.
- `/processes/run`: esecuzione di un processo AI.
- `/processes/convex-hull-preview`: generazione di una maschera convex hull a partire da una maschera esistente.
- `/laboratory-pipelines`: gestione delle pipeline salvate.
- `/laboratory-pipelines/{id}/steps`: gestione degli step associati a una pipeline.

Appendice C

Configurazione e Avvio

C.1 Docker Compose

Il file `docker-compose.dev.yml` permette di avviare l'ambiente di sviluppo quando si desidera eseguire i servizi in container.

C.2 Variabili ambiente

Il backend richiede le variabili ambiente principali elencate di seguito:

- `DATABASE_URL`
- `SUPABASE_URL`
- `SUPABASE_SERVICE_ROLE_KEY`
- `SUPABASE_STORAGE_BUCKET`

La configurazione verifica che il database punti a Supabase Postgres durante l'esecuzione ordinaria.

C.3 Avvio frontend e backend

Il frontend si avvia dalla cartella `frontend/` con i comandi del progetto Vite. Il backend si avvia dalla cartella `backend/` tramite il server FastAPI/Uvicorn, dopo aver installato le dipendenze e configurato il file ambiente.